Large-Batch Training: LR Scaling & Gradient Noise.

RoBERTa is, mathematically, BERT trained *harder*: bigger batches, more data, longer. The single idea that makes a 32× larger batch worthwhile is that averaging more samples cancels noise. Every number on the following pages — the $\sqrt{B}$ noise drop, the linear and square-root learning-rate rules, and a NumPy simulation that confirms them — is computed in full, nothing summarised or deferred.

THE EQUATION

$$\mathrm{Var}(g) = \frac{\sigma^2}{B} \qquad \Longrightarrow \qquad \mathrm{std}(g) = \frac{\sigma}{\sqrt{B}}$$

Worked example. Per-sample gradients drawn from $\mathcal{N}(2, 3^2)$; batch sizes $B \in \{1, 16, 256, 8192\}$, and the $256 \to 8192$ $(32\times)$ jump RoBERTa-style.

Topic 1 of 2 in this module · companion to the course page · v1.0

Read top to bottom: each step feeds the next.

Contents

1 The claim: a minibatch gradient is a sample mean

In stochastic gradient descent we never see the true gradient ∇L. We estimate it from a minibatch of B examples. If g_i is the gradient computed from a *single* example i, the minibatch gradient is their average:

$$\boxed{g = \frac{1}{B}\sum_{i=1}^{B} g_i} \tag{1}$$

We treat the g_i as independent, identically distributed (i.i.d.) random vectors with mean equal to the true gradient $\mu = \nabla L$ and per-sample variance σ^2 (per coordinate). The whole theory of large-batch training falls out of asking: *how noisy is g, and how does that noise change with B?*

Symbol	Meaning	Shape / value
g_i	gradient from one example	vector in $\mathbb{R}^d$
B	batch size (# examples averaged)	scalar
g	minibatch gradient estimate	vector in $\mathbb{R}^d$
$\mu = \nabla L$	true (full-data) gradient	vector in $\mathbb{R}^d$
σ^2	variance of one coordinate of g_i	scalar
η	learning rate (LR)	scalar

A single example's gradient is a wild, jittery arrow — it points roughly downhill but is contaminated by the quirks of that one example. Averaging B such arrows keeps the shared

"downhill" component (signal) while the random jitter partly cancels (noise). More arrows ⇒ a steadier average ⇒ you can trust it enough to take a bigger step.

2 The derivation: mean and variance of the estimate

2.1 The estimate is *unbiased*: $\mathbb{E}[g] = \mu$

Expectation is linear, so

$$\mathbb{E}[g] = \mathbb{E}\left[\frac{1}{B}\sum_{i=1}^{B} g_i\right] = \frac{1}{B}\sum_{i=1}^{B}\mathbb{E}[g_i] = \frac{1}{B}\cdot B\,\mu = \mu.$$

The batch size does **not** change *where* the estimate points on average — only how tightly it concentrates.

2.2 The variance shrinks as $1/B$

For independent variables, variance adds. With $\mathrm{Var}(g_i) = \sigma^2$,

$$\mathrm{Var}(g) = \mathrm{Var}\left(\frac{1}{B}\sum_{i=1}^{B} g_i\right) = \frac{1}{B^2}\sum_{i=1}^{B}\mathrm{Var}(g_i) = \frac{1}{B^2}\cdot B\,\sigma^2 = \frac{\sigma^2}{B}.$$

Taking the square root gives the quantity we actually feel during training — the *noise standard deviation*:

$$\boxed{\mathrm{std}(g) = \frac{\sigma}{\sqrt{B}}} \tag{2}$$

The noise falls off as $1/\sqrt{B}$, *not* $1/B$. To halve the noise you must *quadruple* the batch. This sub-linear payoff is the central tension of large-batch training.

A minibatch gradient is an unbiased estimate of the true gradient whose noise standard deviation is $\sigma/\sqrt{B}$. Bigger batch ⇒ proportionally cleaner gradient ⇒ each step is more trustworthy, so the learning rate can be raised.

3 Worked example: B from 256 to 8192

Take $256 \to 8192$, the kind of jump RoBERTa makes over BERT. The ratio is

$$\frac{B_{\text{new}}}{B_{\text{old}}} = \frac{8192}{256} = 32.$$

By Eq. (2) the noise std drops by $\sqrt{32}$:

$$\frac{\mathrm{std}_{\text{old}}}{\mathrm{std}_{\text{new}}} = \frac{\sigma/\sqrt{256}}{\sigma/\sqrt{8192}} = \sqrt{\frac{8192}{256}} = \sqrt{32} \approx 5.657.$$

So a $32\times$ bigger batch buys only a $5.657\times$ cleaner gradient. How much should the LR grow to keep training stable? Two rules are used in practice.

3.1 The two scaling rules

$$\textbf{Linear rule: } \eta \propto B \qquad \textbf{Square-root rule: } \eta \propto \sqrt{B} \tag{3}$$

- **Linear rule ($\eta \propto B$).** Keeps the *expected update per example* constant. Derivation: with momentum/SGD the total displacement over B examples is $\eta\, g$; matching it to B single-example steps of size η_0 gives $\eta = B\,\eta_0$. Works well at moderate batch sizes with a warm-up; favoured for SGD-style optimisers (the original BERT/large-batch CNN recipe).
- **Square-root rule ($\eta \propto \sqrt{B}$).** Keeps the *noise scale of the update* $\eta \cdot \mathrm{std}(g) = \eta\,\sigma/\sqrt{B}$ constant. Setting this constant forces $\eta \propto \sqrt{B}$. Favoured for adaptive optimisers (Adam/LAMB, used to train RoBERTa) where the update is already normalised, and at very large batches where the linear rule over-shoots.

For our 32× jump:

$$\text{linear: } \eta \times 32, \qquad \text{sqrt: } \eta \times \sqrt{32} \approx \eta \times 5.657.$$

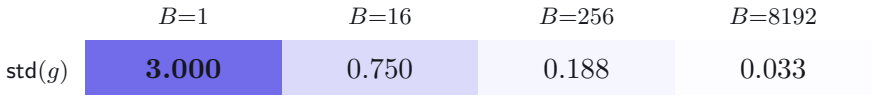
Notice the sqrt rule's LR increase *equals* the noise-reduction factor — by design.

3.2 Simulation: empirical noise matches $\sigma/\sqrt{B}$

We draw per-sample "gradients" from $\mathcal{N}(\mu, \sigma^2)$ with $\mu = 2.0,\ \sigma = 3.0$, form 200,000 independent batch means for each B, and measure the empirical standard deviation of the batch-mean. It tracks the theory to three decimals.

B	empirical mean	empirical std	theory $\sigma/\sqrt{B}$
1	2.0029	2.9998	3.0000
16	1.9981	0.7492	0.7500
256	1.9994	0.1872	0.1875
8192	2.0000	0.0331	0.0332

The mean stays pinned at $\mu = 2$ (unbiased, Sec. 2.1) while the std collapses along the $1/\sqrt{B}$ curve. The heatmap below shades each noise std relative to the loudest case (B=1, capped at `indigo!80`); the bar all but vanishes by B=8192.



4 Properties / consequences that matter

1. **Diminishing returns.** Noise $\propto 1/\sqrt{B}$: doubling B cuts noise by only $\approx 29\%$ $(1 - 1/\sqrt{2})$. Past a problem-dependent "critical batch size" extra examples buy almost no statistical benefit, only compute.

2. **Unbiasedness is free.** The estimate points the right way for *any* $B \geq 1$; batch size is purely a variance (and therefore stability/speed) knob.
3. **LR and batch are coupled.** A bigger batch is wasted unless the LR rises to take a correspondingly bigger step — otherwise you have a clean gradient and a timid step.
4. **Warm-up is essential.** At step 0 the model is random and the large-batch + large-LR combination diverges; a linear LR warm-up over the first few thousand steps fixes this. RoBERTa uses exactly such a warm-up.

> "Bigger batch ⇒ proportionally less noise" is *wrong*. Variance falls as $1/B$ but *noise* (the std) falls only as $1/\sqrt{B}$. Going $256 \to 8192$ ($32\times$) cleans the gradient by $\sqrt{32} \approx 5.66\times$, not $32\times$. Apply the *linear* LR rule blindly at huge batch and the steps become too aggressive for the modest noise reduction — training spikes or diverges. The square-root rule (or LAMB-style per-layer adaptation) is the safer choice there.

5 Where this sits in the track

RoBERTa (*Robustly optimized BERT approach*) keeps BERT's architecture but changes the *training*: $\sim 8\times$ larger batches with LR re-scaled, $10\times$ more data, dynamic masking, no Next-Sentence-Prediction objective, and much longer training. This topic justifies the batch/LR half of that recipe. Topic 2 covers the representation-learning side — contrastive objectives (InfoNCE) used to turn such encoders into Sentence-BERT-style retrieval models for CineMatch semantic search.

A Reproducible (NumPy)

Inputs: $\mu = 2.0$, $\sigma = 3.0$, $N = 200{,}000$ batch-mean draws per B, seed 42. The snippet reproduces every number in the table and the $\sqrt{32}$ factor.

```
import numpy as np
np.random.seed(42)
mu, sigma, N = 2.0, 3.0, 200_000
for B in [1, 16, 256, 8192]:
    means = np.random.normal(mu, sigma, size=(N, B)).mean(axis=1)
    print(B, round(means.mean(),4), round(means.std(),4), round(sigma/np.sqrt(B),4))
# 1    2.0029 2.9998 3.0
# 16   1.9981 0.7492 0.75
# 256  1.9994 0.1872 0.1875
# 8192 2.0    0.0331 0.0332
print("32x batch -> noise /", round(np.sqrt(32),3))   # 5.657
print("linear LR x", 32, " | sqrt LR x", round(np.sqrt(32),3))  # 32 | 5.657
```